Deep Learning — Lecture 10

Kostromina Elizaveta

Contents

1 Self-Supervised Learning (SSL) 2

2 GPT (Generative Pre-trained Transformer) 3

3 BERT (Bidirectional Encoder Representations from Transformers) 3

4 Downstream BERT tasks 5

5 SSL for Images 6

6 Contrastive Learning 7

1 Self-Supervised Learning (SSL)

Self-supervised learning (SSL) alternatively called self-supervision.

Motivation

In **supervised learning** we require large amounts of **labeled** data:

$\{(x_i, y_i)\}_{i=1}^{l}$ $\qquad\qquad$ $\frac{1}{l}\sum_{i=1}^{l} L(f(x_i), y_i) \rightarrow \min_f$

Labelling takes a lot of time and usually costs money (Yandex Toloka, Amazon Mechanical Turk, etc).

At the same time there are loads of unlabeled data on the internet that can be easily collected and used.

$\{(x_i)\}_{i=1}^{l}$ $\qquad\qquad$???

In **self-supervised learning** we want to obtain a **strong pretrained (base)** model that can be later fine-tuned on a downstream task (the pretrain task itself is not that important actually).

Key idea: define a pretext task whose labels come for free from the structure of the data itself, **pre-train without manual labels** and then transfer the learned representations to downstream tasks:

- Embeddings (Word2Vec, FastText)
- Auto-encoders
- Contrastive learning
- Language models

Language Modeling (Text)

Let's train a strong base model on a large corpus of unlabeled text (e.g., Wikipedia or Reddit).

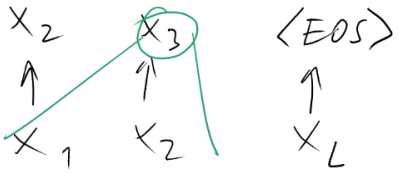
We will define a synthetic task so that the model can learn as much as possible about the structure of language:

- syntax (grammar)
- semantics (meaning)
- dependencies between words

Reminder:
Encoder – reads the input text and produces its representation.
Decoder – generates output text based on this representation.

2 GPT (Generative Pre-trained Transformer)

Architecture: Transformer *Decoder* olny.
Masked self-attention:



Task: language modeling, next-token prediction.

$p(x) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2) \cdots p(x_L \mid x_1, \ldots, x_{L-1})$

GPT is also suited for generation tasks (e.g. ChatGPT)

3 BERT (Bidirectional Encoder Representations from Transformers)

Architecture: Transformer *Encoder* only – **bidirectional** (every token sees all tokens)
Task: *masked* language modeling (task applies not only for texts and this idea often occurs in self supervised learning).

Masked language modeling (MLM)

Goal: learn bidirectional context and capture semantic and syntactic structure of language. Some tokens in the input sequence are randomly masked, and the model is trained to predict them:

output: | Shark | is | a | kind | of | fish | . |

↓ masking

masked-input: | [MASK] | is | a | kind | of | [MASK] | . |

↓ Transformer encoder

prediction: | Tralalero | is | a | kind | of | brainrot | . |

distribution over tokens

The **original tokens are kept** as **target values**.

The occurring issue is the mismatch between pretraining and inference. During training, some tokens are replaced with [MASK] token, which never appears at test time. This can make the model rely on artificial patterns and lead to suboptimal performance when applied to real text.

The solution:
From the input sequence, about **15% of tokens** are randomly selected. Each selected token with probability:

- 80% replaced with [MASK]
- 10% replaced with a random token
- 10% left unchanged

This uncertainty prevents model from overfitting.

Loss computation:
! The loss is computed **only on masked positions** and gradients are computed **only from the loss at masked tokens**, but they propagate through the entire network.

BERT loss:
The overall training objective in BERT is a combination of two tasks:

$$\mathcal{L}_{BERT} = \mathcal{L}_{MLM} + \mathcal{L}_{NSP}$$

MLM:

+ uses both left and right context;
+ learns good contextual representations at the token level
- bad understanding of **long-range coherence** or relationships between sentences in a text

Since MLM has bad performance on long texts, Bert introduces a second task with additional NSP loss.

Next Sentence Prediction (NSP)

Task: to predict if sentence S after T or not (binary classification):
0 – S is **not** the next sentence after T
1 – S **is** the next sentence after T

Let sentence T contain tokens t_1 t_2 ... t_N, sentence S contain s_1 s_2 ... s_M and some tokens in the input sequence are randomly masked:

input: **[CLS]** t_1 t_2 ~~t_3~~ [Mask] $\cdots$ t_N **[SEP]** s_1 s_2 $\cdots$ ~~s_{M-1}~~ [Mask] s_M

BERT is built on a Transformer encoder, and after passing through all self-attention layers, every token gets a contextualized vector.
But since we need **one vector for the whole input**, not one per token, [CLS] token is always added to the input.
A small linear head is placed on top of the final [CLS] vector, and during fine-tuning on other tasks the NSP head is discarded and a new head is placed on top of the same [CLS] vector:

$$h_{[CLS]} \;\rightarrow\; \text{Linear}(d_{\text{model}} \rightarrow 2) \;\rightarrow\; \text{softmax} \;\rightarrow\; \{0,\, 1\}$$

Embeddings:

$$E = E_{\text{token}} + E_{\text{position}} + E_{\text{segment}}$$

$$E_{\text{token}} = E_{[CLS]} + E_{t_1} + E_{t_2} + E_{[MASK]} + \cdots + E_{t_N} + E_{[SEP]} + E_{s_1} + \cdots + E_{s_{M-1}} + E_{s_M}$$

$$E_{\text{position}} = E_0 + E_1 + E_2 + E_3 \cdots + E_N + E_{N+1} + E_{N+2} + \cdots + E_{N+M} + E_{N+M+1}$$

$$E_{\text{segment}} = E_T + E_T + E_T + E_T \cdots + E_T + E_T + E_S + \cdots + E_S + E_S$$

4 Downstream BERT tasks

GLUE (General Language Understanding Evaluation)

Benchmark – a collection of 9 NLP tasks (classification):

Task	Type	What it tests
CoLA	single sentence	corpus of grammatical acceptability
SST-2	single sentence	sentiment (positive / negative)
MRPC	sentence pair	paraphrase detection
STS-B	sentence pair	semantic similarity (regression)
QQP	sentence pair	question paraphrase (Quora)
MNLI	sentence pair	multi-genre Natural Language Inference (NLI)
QNLI	sentence pair	QA-based NLI
RTE	sentence pair	recognising textual entailment
WNLI	sentence pair	Winograd schema (coreference)

Final score – average across all tasks.
(BERT-base achieved 79.6, BERT-large 80.5 and won over GPT at the time).

BERT-like models across modalities

Text:
RoBERTa (*arXiv:1907.11692*), ALBERT (*arXiv:1909.11942*)
Images:
BEiT (*arXiv:2106.08254*), MAE (*arXiv:2111.06377*)
Audio:
HuBERT (*arXiv:2106.07447*)

5 SSL for Images

ImageNet is too good
Several attempts at self-supervised pretext tasks.

Pretext tasks:

- Rotation angle prediction
  (predict whether the image was rotated by 90°, 180°, etc.)
- Jigsaw puzzle
  (shuffle patches of the image; predict the correct permutation)



- Image colorization (grayscale → color)

Moderate success, ImageNet is still king. Let's proceed to something that really works.

Triplet Loss

The goal is to train an embedding function f such that similar inputs are close and dissimilar inputs are far apart in embedding space (Used in e.g. **FaceNet**).

Three elements of a triplet:

- A – anchor
- P – positive (same class as A)
- N – negative (different class)

$$\mathcal{L} = \max\big(\|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 + m,\ 0\big)$$

We want the negative to be farther from the anchor than the positive, with a margin m:

$$m < \|f(A) - f(N)\|_2^2 - \|f(A) - f(P)\|_2^2$$

If this is satisfied, loss is zero, no gradient.
Margin $m > 0$ prevents the model from collapsing all embeddings to a single point (similar to gradient clipping).

6 Contrastive Learning

SimCLR (A Simple Framework for Contrastive Learning of Representations, *arXiv:2002.05709*)

Given a batch $\{x_i\}_{i=1}^N$ from sample $\{x_i\}_{i=1}^L$, apply two **random** augmentations $A(\cdot)$ and $A'(\cdot)$ to each sample:

$$x_i \xrightarrow{A(\cdot)} \tilde{x}_i, \qquad x_i \xrightarrow{A(\cdot)} \tilde{x}_{i+N}$$

This gives a **double batch** $\{\tilde{x}_i\}_{i=1}^{2N}$ of augmented views.

Then:

$$\tilde{x}_i \;\to\; f(\cdot) \;\to\; t_i \;\to\; g(\cdot) \;\to\; z_i$$

where f is the backbone encoder, t_i are the embedings, and g is a small projection head. The head g is **dropped before fine-tuning**; representations t_i are used downstream. We want to maximize agreement between z_i and z_{i+N}.

For each pair (z_i, z_{i+N}) (two views of the same image):

- $z_i, z_{i+N} \to close$ (positive pair)
- z_i, z_j for $j \neq i, j \neq i{+}N \to far$ (negative pairs)

Loss:

$$l(z_i, z_{i+N}) = -\log \frac{\exp(\text{sim}(z_i, z_{i+N}))}{\sum_{k=1}^{2N} \mathbb{1}[k \neq i,\, i{+}N]\, \exp(\text{sim}(z_i, z_k))} \longrightarrow \min_{f,\,g}$$

where $\text{sim}(u, v) = \dfrac{u^\top v}{\|u\|\|v\|}$ is cosine similarity.

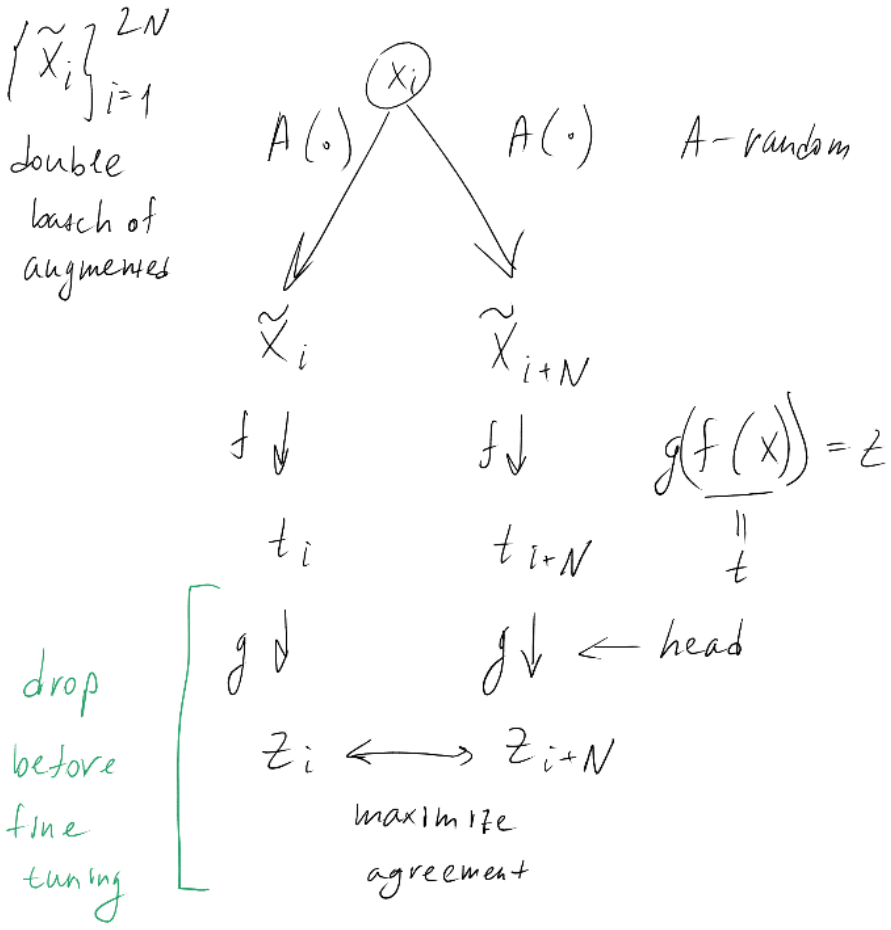
The final loss is symmetrised over both views:

$$L(z_i, z_{i+N}) = \frac{1}{2}\Big(\ell(z_i, z_{i+N}) + \ell(z_{i+N}, z_i)\Big)$$

We need to pick strong augmentations so the model can train:
$(A(x), x)$ – images are still similar
$(A(x), A'(x))$ – images are more distant!

Augmentations $A(\cdot)$**:**

- Random crop
- Color jitter
- Convert to grayscale
- Gaussian blur


$\{\tilde{x}_i\}_{i=1}^{2N}$
double batch of augmentes
x_i
$A(\cdot)$
$A(\cdot)$
A - random
$\tilde{x}_i$
$\tilde{x}_{i+N}$
$f \downarrow$
$f \downarrow$
$g(f(x)) = z$
t_i
t_{i+N}
t
$g \downarrow$
$g \downarrow$ ← head
$z_i \longleftrightarrow z_{i+N}$
maximize agreement
drop before fine tuning